

Python Oops Question -1

Q1. Create a class `Animal` with `make_sound()` and derived classes `Dog`, `Cat`, `Cow` that override it.

Demonstrate polymorphism by iterating over a list of different animal objects and calling `make_sound()`.

Q2. Write a function `operate(device)` that calls `device.start()`.

Pass in objects of `Car`, `Computer`, and `WashingMachine` — all of which define a `start()` method, but share no inheritance relationship.

Show that Python's polymorphism works through *behavior, not type*.

Q3. Create a `Vector` class that supports:

- `+` operator → add coordinates
- `==` operator → compare equality

Show how operator overloading gives natural polymorphism to user-defined classes.

Q4. Create a base class `Transport` with `move()` and derived classes `Bus` and `Bike` that override it but also call the parent implementation using `super()`.

Show the combination of reuse + custom behavior.

Q5. Using the `abc` module, create an abstract class `Notification` with `send()`.

Implement subclasses `EmailNotification`, `SMSNotification`, `PushNotification` — each with its own `send()` logic.

Demonstrate polymorphism by looping over all and calling `send()`.

Q6. Design:

- Base class `Payment` with `process(amount)`
- Subclass `CreditCardPayment` adds `process(amount, card_type)`

Demonstrate what happens when overriding with different signatures and how Python handles it.

Q7. Create:

- Class `Sorter` with `change(strategy)` method. Separate strategy classes: `BS`, `MS`, `QS`, each implementing a different logic method.

Demonstrate how polymorphism can be achieved without inheritance by using interchangeable strategy objects.

Q8. Create:

- Base `Account` → `withdraw()`
- Subclass `SavingsAccount` → modifies `withdraw()`

- Subclass `PremiumSavingsAccount` → overrides again but calls parent using `super()`
Show how polymorphism works across multiple levels.

Q9. Create a function `draw(shape)` that works for objects of classes `Circle`, `Square`, and `Rectangle`,

each implementing a `draw()` method.

Add another unrelated class `Car` with `draw()` and pass it — what happens and why?

Q10. Design a polymorphic system for payment handling (`UPI`, `Card`, `Cash`) — all have a `pay()` method.

Now implement a version that checks types explicitly using `isinstance()` before calling `pay()`.

Compare both designs and explain why one breaks the spirit of polymorphism.